

	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios					CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:			
	Nombre y Apellidos:							

Ejercicios de repaso Programación de Servicios

1º parte: Programación base (repaso 1º en C#).

1.1.

Se desea realizar un gestor de cuerpos celestes en consola. Para ello se pide generar el siguiente esquema de clases:

a) **Clase Astro** con propiedades privadas: nombre (String) y radio.

Al guardar el nombre lo hará en mayúsculas y eliminando posibles espacios en los extremos del nombre.

Habrà una sobrecarga del get con un parámetro tipo carácter que devuelve el nombre con sus letras separadas por ese carácter. Por ejemplo si se llama getNombre('_') a un astro denominado "SOL" devuelve "S_O_L".

Al guardar el radio se comprueba que sea positivo, si no fuera así lanza la excepción RadioNegativoException creada por ti.

Un constructor que inicialice las propiedades con parámetros.

Sobreescribe Equals de forma que se considera que dos astros son iguales si son iguales sus nombres y además son de la misma clase.

Sobreescribe ToString de forma que devuelva el nombre según getNombre y el radio con 2 decimales.

b) **Clase Planeta** que hereda de Astro y dispone de una propiedad privada booleana denominada gaseoso con set y get.

Otra propiedad pública que será una colección de Astros que serán los satélites.

Tendrá dos constructores, uno que inicialice los miembros nombre, radio y gaseoso con parámetros y la colección de satélites vacía. Usa para las inicializaciones una llamada al constructor de la clase base.

El otro constructor sin parámetros que inicializa a "" nombre, 0 radio y false la propiedad gaseoso llamando al primer constructor.

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

c) En el programa principal (Estará en la clase **Program**) se crea una colección de Astros y el siguiente menú:

- Añade Planeta: Pregunta si es gaseoso y pide el nombre y radio. También pregunta cantidad de lunas y se introducirán sus nombres y radios.
- Añade otro astro: Pide su nombre y radio.
- Mostrar datos: Muestra toda la colección detectando si es Astro o Planeta para mostrar todos sus datos. En el caso de Astro simplemente llama a ToString() y lo muestra, en el caso de Planetas muestra todos los datos y en particular llamando a getNombre con parámetro `.`
- Elimina repetidos. Busca Astros/Planetas iguales y elimina todas las apariciones menos la primera.
- Salir. Se mantiene en el menú hasta que se selecciona esta opción.

1.2.

En un instituto se desea realizar una serie de estadísticos con las notas de los alumnos. Para ello, y como fase inicial de un futuro proyecto, se pide la realización de un programa de simulación de notas de dicho instituto. Para rellenarla se usarán notas aleatorias entre 0 y 10 sin decimales.

Se debe crear a continuación un menú (estará en la clase *Usuario* como se explica más abajo) con las opciones:

- Visualizar tabla completa
- Calcular la media de notas de toda la tabla.
- Media de un alumno
- Media de una asignatura
- Visualizar notas de un alumno
- Visualizar notas de una asignatura
- Nota máxima y mínima de un alumno

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

- Tabla solo de aprobados

Los nombres de los alumnos estarán en un vector y los nombres de las asignaturas en otro.

Debes estructurar el programa en al menos tres clases:

- **Clase Aula** donde se encuentra el array bidimensional de notas privado , un vector de string para los nombres de alumnos y otro para los nombres de asignaturas. Estarán aquí todos los métodos de proceso de datos (obtención de medias, devolución de una fila, etc.).
- En concreto haz una función única que coloque el máximo y el mínimo de un alumno en parámetros por referencia. No devolverá nada.
- Tendrá un constructor al cual se le pasan dos vectores: de nombres de alumnos y otro de nombres de asignaturas. El primero establecerá el número de filas de la tabla y el segundo el de columnas, dicha tabla se rellenará con notas aleatorias entre 1 y 10.
- La función que devuelva la tabla de aprobados hazla que devuelva un hashtable o dictionary usando como clave el nombre del alumno y como valor un vector con las notas del mismo. Contendrá solo alumnos con todas las materias aprobadas.
- Las medias se devuelven siempre como doubles.
- No debe tener interfaz de usuario.
- **Clase Usuario** donde tendrás un objeto tipo Aula y se organizará todo el interfaz de usuario para la aplicación. El menú principal estará en una función de esta clase.
- **Clase Program:** Una tercera clase con el main que simplemente cree un objeto del tipo Usuario y ejecute su método principal que contendrá el menú.

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

1ª Evaluación

Tema Archivos/lambdas/delegados.

Ejercicio 2.1

Ampliar el programa gestor de Astros (Ejercicio 1.1) de forma que guarde la información de la base de datos que se está creando.

La colección de Astros debe cargarse al iniciar el programa y guardarse en el disco duro al finalizar el mismo de forma transparente al usuario (el usuario no participa). Esos son los únicos cambios en el programa, al inicio y fin del mismo, no cambios nada más. Hazlo en la carpeta %APPDATA% del usuario.

Ejercicio 2.2

Crea estos dos vectores en un Main:

```
int[] notas = { 5, 2, 8, 1, 9, 4 };
```

```
string[] palabras = { "Sol", "Luna", "Estrella", "Cielo" };
```

Usa métodos de la clase Array junto con lambdas para obtener directamente estos elementos del vector notas:

- Saber de si hay algún aprobado (Si existe o no) en notas.
- Mostrar los aprobados de notas.
- Indicar la posición en el array del último aprobado
- Mostrar la nota del último aprobado.
- Cuanto tienen nota par.

Y ahora haz lo siguiente con el vector palabras:

- Cual es la primera palabra de más de 3 caracteres.
- Mostrar todas las palabras en mayúsculas.
- Indica la posición de la primera palabra que empieza por E.

	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios				CURSO: 2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:	
	Nombre y Apellidos:					

Ejercicio 2.3

a) Desarrolla una función para la realización de cualquier menú mediante delegados. Dicha función se denominará **MenuGenerator** y tendrá dos parámetros: El primero un vector de strings con los nombres de las opciones. El segundo un vector de delegados (hazlos que devuelvan void y sin parámetros y el nombre del delegado que sea MyDelegate) que serán las funciones que correspondan a cada opción del menú.

Al ejecutar MenuGenerator saldrá un menú clásico con las opciones indicadas por el primer vector y una última opción Exit (independientemente del número de entradas del menú). Si se introduce una opción no válida (fuera del rango) dará un mensaje de error y vuelve a pedirlo. Solo finaliza el menú si se selecciona la última opción, que será la de salir.

MenuGenerator debe ser robusto ante parámetros no válidos de forma que devolverá true si el menú se crea correctamente y false si ha habido algún problema con algún parámetro (longitudes de vectores distintas, parámetro null,...)

Para probarlo utiliza las siguientes funciones y el siguiente programa principal:

```
static void f1()
{
    Console.WriteLine("A");
}

static void f2()
{
    Console.WriteLine("B");
}

static void f3()
{
    Console.WriteLine("C");
}

static void Main(string[] args)
{
    MenuGenerator(new string[] { "Op1", "Op2", "Op3" },
                  new MyDelegate[] { f1, f2, f3 });
    Console.ReadKey();
}
```

b) Elimina las funciones f1, f2 y f3 y pásalas al array MyDelegate como expresiones lambda.

	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios					CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:			
	Nombre y Apellidos:							

Tema hilos

Ejercicio 3.1. Ejercicios de entrenamiento de técnicas base de hilos.

a) Lanzamiento de hilos, uso de lock, uso de sleep, uso de Join.

Crea un programa en el que se lancen 2 hilos.

Uno que muestre en una parte de la consola (arriba a la izquierda) 50 múltiplos de un número que se le pasa como parámetro (el parámetro de la función es int). Entre múltiplo y múltiplo hay una pausa (sleep) de 300ms.

Otro que muestre en otra parte de la consola (arriba a la derecha) numero primos menores que un parámetro (el parámetro de la función es object). Entre número y número hay una pausa de 200 ms.

El Main espera con un Join a ambos hilos.

b) Uso de while-lock-if

Copia el programa anterior en otro proyecto y modifícalo para que en el momento que un hilo termine también acabe el otro.

c) Uso de IsBackground

Copia el programa del apartado (a) en otro proyecto y modifícalo para que el main en vez de esperar con join, espere con un sleep de 3 segundos. Una vez que acabe el sleep, debe finalizar el programa completo.

Ejercicio 3.2

Crea dos hilos (Thread) compitiendo que traten de incrementar (el primer hilo) o decrementar (el segundo hilo) en 1 unidad una variable que comienza en 0. Funcionarán de forma continua hasta que llegue a 500 (primer hilo) o a -500 (segundo hilo). Además se mostrará en pantalla la variable cada vez que cambie indicando quién la ha cambiado (thread 1 o thread 2 y cada uno en un color). Ambos hilos deben parar en cuanto uno consiga su objetivo. No uses setCursorPosition.

El Main, una vez que lanza los hilos, se queda en espera hasta que ambos hilos finalizan, luego informa de cual ha ganado.

	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios					CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:			
	Nombre y Apellidos:							

Ejercicio 3.3

Realiza en consola el juego de carreras de caballos. Habrá un archivo en %APPDATA% con los nombres de los caballos separados por punto y coma (;). La cantidad de nombres indicará la cantidad de caballos que correrán. Si hay más de 10 nombres se limitará a 10.

Al comienzo se le pide el nombre y la edad al usuario y, si tiene 18 años o más este hace su apuesta. Si no cumple la edad la aplicación termina.

Luego empieza la carrera de caballos de forma que cada uno se mueve una distancia aleatoria y "duerme" un tiempo aleatorio. Al empezar la carrera, el Main se queda en espera. Una vez que un caballo llega a la meta todos paran y el Main continúa indicando el caballo ganador y si el usuario ha ganado.

Además añade en un archivo los datos de Nombre, edad el caballo por el que apostó y si ganó o no el usuario.

Se permitirá la repetición del juego. Si el usuario decide no repetir, se mostrará en pantalla los datos de los ganadores que están guardados en el archivo

Nota: De cara a realizar pruebas de este juego, se recomienda quitar la aleatoriedad temporalmente para forzar a que varios caballos lleguen a un tiempo y ver que solo uno es el que "cruza" la meta.

Ejercicio 3.4

Realiza un simulador de descargas asíncronas en modo gráfico. Debes realizar una función Task que devuelva un string con el contenido supuestamente descargado. La cabecera de la función tendrá esta forma:

```
async Task<string> DownloadFileAsync(string fileName, int delayMs)
```

En realidad se simulan los tiempos usando Task.Delay para el retardo. Lo que realmente va a devolver la función es el mensaje "File *fileName* downloaded in *delay ms*". Por supuesto *fileName* y *delay* serán los valores de los parámetros.

La aplicación WindowsForms dispondrá de un TextBox denominado txtFileName y un botón. Cada vez que se meta ahí una cadena y se pulse el botón o se pulse la tecla Enter se lanza una tarea con DownloadFileAsync usando el nombre de txtFileName y un retardo aleatorio entre 1 y 10 segundos. Cuando finaliza se añade la cadena devuelta a una nueva línea de otro TextBox multilinea (txtResults).

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

Por supuesto el programa no debe bloquearse nunca.

Ejercicio 3.5

Realiza una aplicación a la cual se le pueda introducir una ruta o URL en un textbox (**txtUrl**) y disponga de los siguientes botones con las posibilidades indicadas:

Búsqueda: Este botón tiene asociado un TextBox donde se puede meter una cadena de caracteres. Al pulsarlo el programa buscará todas las apariciones de la cadena en todos los archivos *.txt que haya en el directorio indicado por txtUrl.

Para ellos sigue los siguientes pasos:

1. Haz una función denominada BuscaPalabra a la que le pasas una ruta y una cadena y busque la cantidad de veces que aparece la cadena en el archivo. Devuelve una cadena donde se indica el nombre del archivo, la palabra buscada y la cantidad de veces que aparece.

No hagas un algoritmo óptimo de búsqueda con split o expresiones regulares ya que se trata de ver que aunque la búsqueda sea lenta el programa sigue en funcionamiento. Haz una comparación exhaustiva del string barriendo cada substring del archivo. Incluso si lo consideras mete retardos.

2. En el click del botón haz un bucle que recorra todos los archivos de texto del directorio y con cada uno lanza una tarea (Task.Run) que ejecute la función BuscaPalabra con cada archivo y añádela una colección de tareas (List<Task<string>>).
3. Haz un bucle que mientras haya tareas en la colección, espere a que acabe una (WhenAny), cuando esto suceda la eliminas de la colección y cojes el resultado de esa tarea y lo añades al ListBox. Como las otras tareas siguen en ejecución en la siguiente vuelta del bucle se espera a la siguiente.

Posición: Este también tiene asociado un TextBox (si quieres puede reutilizar el de búsqueda). Este indica la posición donde aparece la palabra indicada en el TextBox en cada uno de los archivos del directorio. Si no existe pone -1 un el ListBox de resultados. La diferencia que este hace todas las búsquedas y luego pone todos los resultados de golpe (WhenAll).

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

Haz en este caso una función que busque la primera aparición de una palabra y devuelva una cadena indicando Archivo, palabra y posición o que no existe (mete un Delay aleatorio para ralentizarla). Nuevamente evita optimizar algoritmos o incluso si quieres puedes introducir retardos.

Http: Este también tiene asociado un TextBox (si quieres puede reutilizar el de búsqueda). Este utiliza la clase HttpClient y muestra en la listbox lo que devuelve la función GetAsync pasándole como parámetro la URL que haya en el textbox.

Las entradas en el ListBox deben indicar: Botón pulsado y resultado.

Se permite la captura de excepciones genéricas en este ejercicio.

Puedes descargar textos largos de webs como : <https://www.gutenberg.org>

	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios					CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:			
	Nombre y Apellidos:							

2ª Evaluación

Ejercicio 4.1

Realiza un servidor multihilo de fecha y hora. Aceptará los siguientes comandos desde cualquier cliente (Protocolo):

- **time:** Envía hora, minutos y segundos,
- **date:** Envía día, mes y año.
- **all:** Envía tanto la fecha como la hora.
- **close password:** Junto con el comando close se debe verificar que viene una contraseña. Si esta es correcta el servidor ha de finalizar y se lo indica al cliente. Si no devuelve un mensaje de error al cliente (Debe diferenciarse el error de contraseña no válida o que no se haya enviado la contraseña).

La contraseña estará guardada en un archivo de texto en %PROGRAMDATA% denominado password.txt con una sola línea conteniendo la contraseña en texto plano.

En cuanto se envíe el dato pedido, se cerrará la conexión (No hay bucle de atención al cliente en el hilo del cliente).

Ejercicio 4.2

Realización del servidor de una sala de chat. Los clientes serán consolas telnet. Se pide:

Crea una clase denominada Sala. En dicha clase habrá al menos la propiedad clientes que será una colección de Sockets o Streamwriters y los siguientes métodos:

leePuerto(): método que trata de leer el archivo %PROGRAMDATA%\puerto.txt y lee el valor que haya en su primera línea. Si es un valor de puerto válido lo devuelve. Si no lo es o no puede abrir el archivo devuelve 31416.

envioMensaje(string m, IPEndPoint ie): Este método recorre la colección clientes y les envía el mensaje parámetro m a cada uno de ellos. Antepone al mensaje la IP y

	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios				CURSO: 2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:	
	Nombre y Apellidos:					

el puerto de quién lo envía. Además el mensaje no debe repetirse en el cliente que lo ha enviado (identificado por el segundo parámetro).

`iniciaServicioChat()`: Método principal donde se inicia el servicio. Realiza la programación necesaria para la comunicación por red. Se obtiene el puerto llamando a `leePuerto`, si el puerto está ocupado lo incrementará en una unidad de forma sucesiva hasta que encuentre uno libre. Si llega a máximo puerto permitido se vuelve a poner en 10000. Informa por pantalla del puerto de conexión y se queda a la escucha.

Finalmente entra en un bucle en el cual se realiza la conexión con el cliente iniciando un hilo que ejecuta la función `hiloCliente`. Además se añadirá el socket de cliente o el `Streamwriter` a la colección clientes.

`hiloCliente(object socket)`: Función que se ejecuta como hilo según se ha comentado. El parámetro es el socket de cliente. Indicará por pantalla la IP y puerto del cliente. Luego mandará un mensaje de bienvenida al cliente y le informa de la cantidad de clientes conectados a la sala de chat.

A continuación queda a la espera de el cliente le envíe algún mensaje. Si sucede esto, recoge el mensaje y lo reenvía al resto de los clientes llamando a la función `envioMensaje`.

Si el mensaje que envía el cliente es la palabra MELARGO se cerrará la conexión con dicho cliente y se eliminará de la colección.

En el programa principal que estará en la clase `Program` simplemente se crea un objeto del tipo `Sala` y se llama a la función `iniciaServicioChat`.

Ejercicio 4.3

Ahorcado en red. Se trata de realizar un servidor de palabras y récords para un posible juego del ahorcado (En principio no harás el cliente). La especificación del servidor será la siguiente:

Dispone de acceso a un archivo de palabras (archivo de texto con palabras separadas por comas). Cuando lea el archivo pasa las palabras a mayúsculas.

También mantiene un archivo de récords, que será un archivo binario donde se guarda una lista con 3 objetos tipo `Record` (clase sencilla que almacena nombre de

	RAMA:	Informática	CICLO:	DAM			
	MÓDULO	Programación de servicios				CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:		
	Nombre y Apellidos:						

3 caracteres y cantidad de segundos)

Se dispone del siguiente protocolo de comandos que puede recibir el servidor:

gw: GetWord. El servidor envía una palabra aleatoria al cliente de la lista que tiene.

sw palabra: SendWord. El cliente le envía una palabra nueva al servidor. Este la guarda en el archivo y la añade a la colección de palabras actual. El servidor envía un OK o ERROR si se ha hecho todo bien o hubo un problema respectivamente.

gr: GetRecords. El servidor le envía la lista de récords del archivo al cliente.

sr record: SendRecord. El cliente le envía un nuevo récord al servidor. Si el tiempo es menor que alguno que los que tiene almacenado lo guarda y, si fuera necesario porque ya hay tres, deshecha el peor. Si no fuera menor que ninguno lo deshecha. El servidor responde con un ACCEPT o REJECT dependiendo si ha guardado o no el récord.

close clave: cierra el servidor si se dispone de la clave adecuada.

Es un protocolo sin conexión, es decir, en cuanto el servidor responde, finaliza la conexión.

Ejercicio 4.4 (Ejemplo examen)

A principios del S.XXI la corporación FlyRell llevó la evolución de las moscas a la fase Flyxus. Un ser casi idéntico al insecto original conocido como un replicantfly. Los replicantfly 6 eran superiores en fuerza y agilidad a sus congéneres biológicos y encima con una picadura de mil demonios. Fueron declaradas ilegales en la tierra salvo en Guantánamo y en las nuevas ediciones de Gran Hermano (vaya usted a saber con qué objetivo). Las fugas existían y surgió un escuadrón especial de la policía denominado Fly Runners. Pero era un trabajo duro y cuando les soltaron por ahí a matar moscas estos dijeron a sus superiores: "Mira bro, sin entrenar yo paso de perseguir bichos".

Las instancias superiores de fuerzas y seguridad del estado encargaron un sistema completo de entrenamiento en red para los Fly Runners a los fabulosos alumnos del Colegio Vivas, bien sabida su legendaria capacidad de codificación que dejarían a Neo de noob total. Así que, bro, esto es lo que tienes que hacer:

	RAMA:	Informática	CICLO:	DAM				
	MÓDULO	Programación de servicios					CURSO:	2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:			
	Nombre y Apellidos:							

Se desea realizar un servidor de moscas, para ello se creará un nuevo proyecto de consola en Visual Studio con las siguientes características:

Habrà una clase denominada **FlyRunner** la cual tendrá las siguientes características:

- Propiedad **Sw** de tipo StreamWriter.
- Propiedad **KilledFlies** de tipo int. Representa el número de moscas matadas por un FlyRunner.
- Propiedad **Bites** de tipo int. Representa el número de picaduras recibidas por un FlyRunner.
- Constructor con un único parámetro tipo StreamWriter que inicialice sw. Los int los inicializa a 0.

Otra clase denominada **FlyServer** con los siguientes miembros:

- Colección de objetos tipo FlyRunner que representará a los clientes conectados. Se denomina **clients**.
- Método **GetPort()**: función que trata de leer un archivo binario (mira el apéndice en el tema de archivos) con números enteros denominado *port.bin* situado en el directorio indicado por la variable de entorno *userprofile*. Debe leer dos números enteros seguidos que corresponden a un puerto principal y uno secundario.

Si el archivo no existe, hubiera algún problema o los números son valores no válidos para puertos, creará un archivo binario con los valores 135 y 31416 como enteros en las dos primeras posiciones y luego escribe 8 números enteros más aleatorios entre 1024 y el valor máximo posible de puerto (escribe todo como int). En este caso la función devuelve -1.

Si ha cargado bien los dos puertos, devolverá el primero de ellos que esté libre (haz comprobación de puerto ocupado aquí). Si ninguno lo estuviera devuelve -1.

- Método **InitServer()**: Este método tiene una estructura clásica de servidor multihilo. Obtiene un puerto mediante GetPort, luego se queda a la escucha en cualquier interfaz de red en el puerto indicado por la función. Si esta función devuelve -1 el servidor finaliza.

COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios				CURSO: 2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:	
	Nombre y Apellidos:					

Si entra un cliente, lo lanza en un hilo aparte llamando a la función `FlyRunnerThread` definida más abajo. Tras esto se queda a la escucha de un nuevo cliente.

- Método **FlyRunnerThread**. Representa al hilo de atención al cliente y recibe como parámetro un **object** que será de tipo `Socket`. Dentro del mismo se realizan las tareas típicas de inicialización de los distintos `Streams` y añadir un objeto tipo `FlyRunner` con el `StreamWriter` actual a la colección *clients*. Tras esto entra en el bucle de comandos que sigue el siguiente protocolo:
 - El servidor le envía una mosca, es decir, el mensaje **fly**, y se queda a la espera de que el cliente responda. Realiza un switch con la recepción de mensajes cuyos casos pueden ser:
 - Posibilidad 1: Mata a la mosca, es decir, incrementa el contador de moscas (`KilledFlies`) de su objeto `FlyRunner` (`fr`) e informa con el mensaje: `$"Killed {fr.KilledFlies} fly/flyes!!"`
 - Posibilidad 2: La mosca le da un mordisco con lo cual incrementa el contador de picaduras de su objeto `FlyRunner` (`fr`) e informa con el mensaje: `$"You have been bitten. Number of bites: {fr.Bites}."`
 - Posibilidad 3: La mosca escapa y pica a otro flyrunner de forma aleatoria (es decir, selecciona uno aleatorio de la colección distinto al actual para incrementar su nº de picaduras). Además se le envía el mensaje: `"Other fly bites you!!"`
 - Si el cliente manda un mensaje **quit** (Terminar) entonces se le informa de los mordiscos recibidos y de las moscas matadas. Además se le indican al resto de los clientes que ha salido alguien mediante el siguiente mensaje (`fr` es el objeto actual, el que ejecutó el `quit`): `$"Someone leaves with {fr.Bites} bites and {fr.KilledFlies} flies killed."`
- Tras este mensaje el hilo terminará de forma natural.
- Si el cliente sale abruptamente (pulsando el icono X en la consola, por ejemplo) el hilo terminará también de forma natural y con el mensaje al resto de los clientes tal y como se explica en "quit". Evita repetir código mediante `goto case`, por ejemplo. Tanto en este caso como en el de "quit"

	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Programación de servicios				CURSO: 2º
	PROTOCOLO:	Ejercicios repaso	AVAL:	2ª	DATA:	
	Nombre y Apellidos:					

el cliente será eliminado de la colección clients.

- Finalmente si hay un error en el comando (es decir, no se escribe nada de lo indicado anteriormente), la mosca le pica dos veces (incrementa en 2 el contador de picaduras) y el mensaje sera: `"Big mistake, you were bitten twice. Number of bites: {fr.Bites}."`. Y además le quitará una picadura a otro FlyRunner aleatorio (puedes estar incluido) y se le manda el mensaje `"I get you one bite!!"`.

Revisa bien las variables comunes entre varios hilos para un correcto uso de los lock.

En la clase **Program** estará el Main que simplemente llama a InitServer de un objeto tipo FlyServer.

Nota final: si te diera tiempo a realizar todos estos ejercicios, realiza el del examen de evaluación y si lo acabas habla con el profesor para que te indique otras tareas de repaso.